

Теория языков программирования и методы трансляции

1 Общая информация по курсу

Это обзор проекта компилятора и критериев его оценки. Не стоит ожидать, что вы поймете все технические термины, поскольку мы еще не изучали их на занятиях. Мы приводим этот обзор только для того, чтобы дать вам представление о том, какой проект мы вам предлагаем. Дополнительные инструкции по каждому этапу будут опубликованы, чтобы предоставить более подробную техническую информацию о проекте. К концу курса вы создадите компилятор, который будет принимать на вход программы на языке **Decaf**, подмножестве языка **Си**, и генерировать на выходе **исполняемый код для архитектуры x86**.

2 Организация обучения

Первый этап (лексический анализ и синтаксический разбор) вы пройдете индивидуально. На последующих этапах вы будете работать в группах по три-четыре человека. По возможности выбирайте себе партнеров самостоятельно. Каждая группа должна написать по одному компилятору для языка **Decaf**. Мы ожидаем, что все группы успешно пройдут все этапы. Начало занятия очень динамичное: не отставайте!

3 Этапы проекта

Этап 1: Лексический анализ и синтаксический разбор

Сканер принимает на вход исходный файл на языке **Decaf** и выводит последовательность лексем в процессе, известном как лексический анализ. Лексемы — это последовательности символов, которые являются основными единицами программ на языке **Decaf**.

Токенами могут быть:

- Оператор (***** или **[**)
- Ключевое слово (**if**, или **while**)
- Идентификатор (**foo**)
- Литерал (**42**, **'a'**, **"hello"**)

Немаркеры (например, пробелы или комментарии) игнорируются. О недопустимых маркерах (незавершенных строковых литералах) необходимо сообщать.

Парсер принимает на вход последовательность лексем и проверяет, соответствуют ли они спецификации языка. Чтобы пройти эту проверку, входные данные должны содержать все соответствующие фигурные скобки, точки с запятой и т. д. Типы, имена переменных и функций не проверяются. На выходе парсер формирует синтаксическое дерево — древовидное представление программы.

Спецификация Decaf — это грамматика языка, которую вам нужно будет разделить на спецификацию сканера и спецификацию синтаксического анализатора. Затем вы реализуете сканер и синтаксический анализатор либо с нуля на выбранном вами языке программирования, либо с помощью генератора сканеров (например, *lex*, *flex*, *jflex*, *antlr*) и генератора синтаксических анализаторов (например, *yacc*, *bison*, *cup*, *antlr*) на ваш выбор.

Отчет

Необходимо будет подготовить краткий отчет, который поможет нам понять, какой подход (т. е. написание кода вручную, использование генератора парсеров, выбор языка программирования) вы выбрали.

Этап 2: Промежуточное представление (Intermediate Representation, IR) и семантическая проверка

Семантический анализатор принимает на вход синтаксическое дерево и проверяет правильность программы, выполняя действия, которые не могут быть выполнены сканером и синтаксическим анализатором.

Например, анализатор проверяет, что переменные объявлены до их использования, что переменные имеют правильные типы и что функции вызываются с правильным количеством аргументов нужного типа.

Семантический анализатор также создаст таблицу символов, в которой будут храниться тип и расположение каждого идентификатора. Полный список проверок приведен в спецификации **Decaf**.

Важно создать таблицу символов, так как без нее вы не сможете собрать генератор кода. Однако полнота проверки не окажет существенного влияния на последующие этапы проекта.

К концу этого этапа фронтенд вашего компилятора готов, и вы разработали промежуточное представление (Intermediate Representation, IR), которое будет использоваться остальной частью компилятора.

Отчет

Вы также подготовите краткий отчет, в котором подробно опишете, как ваша команда планирует распределить работу, связанную с проектом, какие вопросы возникают и на каком этапе находится реализация.

Этап 3: Генерация кода

На этом этапе вы создадите рабочий компилятор, сгенерировав неоптимизированный ассемблерный код для архитектуры x86-64 на основе промежуточного представления, созданного на предыдущем этапе. Поскольку на этот этап у вас относительно мало времени, сосредоточьтесь на корректности работы и не используйте никакие оптимизационные приемы, какими бы простыми они ни были.

Этапы генерации кода следующие: сначала сложная семантика Desaf разбивается на простое низкоуровневое промежуточное представление. Например, такие конструкции, как циклы и условные операторы, преобразуются в сегменты кода с простыми инструкциями сравнения и перехода. Затем промежуточное представление сопоставляется с прикладным двоичным интерфейсом (Application Binary Interface, ABI), то есть соглашением о вызовах и использовании регистров и стека.

Затем генерируется соответствующий машинный код x86-64. Наконец, код, структуры данных и хранилище данных преобразуются в формат ассемблера. Мы расскажем об объектном языке. Созданный с его помощью объектный код будет запущен на тестовой машине.

Отчет

Вы сдадите проектную документацию, в которой будут описаны технические детали этапов 1, 2 и 3.

Этап 4: Оптимизация потока данных

Этот этап включает в себя создание фреймворка для управления потоками данных, который поможет оптимизировать код, сгенерированный вашим компилятором. На этом этапе вам необходимо реализовать фреймворк для управления потоками данных и проходы для оптимизации потоков данных, чтобы протестировать фреймворк. Этот фреймворк будет использоваться на заключительном этапе для создания проходов для оптимизации потоков данных.

Описание фреймворка и необходимых оптимизаций будет представлено в следующем раздаточном материале.

Отчет

Вы предоставите проектную документацию, в которой будут описаны технические детали четвертого этапа.

Этап 5: Другие оптимизации

Заключительный этап — это масштабная открытая фаза. На этом этапе задача вашей команды — сгенерировать оптимизированный код для программ, чтобы они выполнялись корректно и в кратчайшие сроки.

Существует множество различных оптимизаций, которые можно применить для улучшения сгенерированного кода. Вы можете выполнять оптимизацию потока данных, например распространение констант, устранение общих подвыражений, распространение копий, перемещение инвариантного кода цикла, удаление недостижимого кода, удаление мертвого кода и многое другое, используя фреймворк, созданный в предыдущем разделе.

Вы также можете реализовать планирование команд, распределение регистров, оптимизацию «смотрового окна» и даже распараллеливание на несколько ядер целевой архитектуры.

Для определения и приоритизации оптимизаций вам будет предоставлен набор тестов для нескольких простых приложений. Ваша задача — проанализировать эти программы и, возможно, оптимизировать их вручную, чтобы определить, какие оптимизации окажут наибольшее влияние на производительность. В отчете необходимо подробно описать процесс, который вы использовали для определения и обоснования реализованных оптимизаций.

Отчет

Вы предоставите проектную документацию с техническими подробностями пятого этапа.

4 «Компиляторные гонки»

Последним занятием станут «Компиляторные гонки», в ходе которого ваша группа будет соревноваться с другими группами в определении компилятора, который генерирует самый быстрый код.

5 Оценка

На каждом этапе необходимо предоставить проектную документацию и полный исходный код (включая все файлы, необходимые для сборки проекта). Ваши проекты будут загружаться через **GitHub**. Не включайте скомпилированные файлы. Вместо этого в репозитории должен быть исполняемый файл **build.sh** в корневом каталоге, который скомпилирует ваш код. Эти файлы включены в шаблонный код; при необходимости вы можете их изменить.

Этапы со 2-го по 5-й будут проходить в группах. Каждая группа получит доступ к репозиторию своего проекта на **Github**.

Существует несколько ограничений на структуру проекта, за исключением того, что он должен быть автономным (за исключением разрешенных библиотек и среды программирования) и содержать исполняемые файлы **build.sh** и **run.sh** в корневом каталоге.

Мы будем публиковать общедоступные тесты, связанные с каждым этапом, вместе с инструкциями по прохождению каждого этапа. Также будут доступны закрытые тесты, которые будут публиковаться после завершения каждого этапа. После прохождения каждого этапа мы будем оценивать ваши работы по результатам как общедоступных, так и закрытых тестов.

Эскизный проект будет учитываться при выставлении оценки за каждый этап. Пожалуйста, не пренебрегайте его сдачей.

6 Справочник по командной строке

Ваш компилятор должен иметь следующий интерфейс командной строки.

Опция	Описание
-t --target <stage>	<stage> — это сканирование, синтаксический анализ, промежуточный код или трансляция. Компиляция должна проходить через все эти этапы.
-o --output <outname>	Записать вывода в <outname>
-O --opt [optimization,...]	Выполните перечисленные (через запятую) оптимизации. <all> означает все поддерживаемые оптимизации. <optimization> удаляет оптимизации из списка
-d --debug	Вывести отладочную информацию. Если этот параметр не указан, при успешной компиляции на экран ничего не выводится.

Аргументы командной строки, которые необходимо реализовать, перечислены в таблице выше. Необходимо указать ровно одно имя файла, которое не должно начинаться с дефиса. Имя файла не должно указываться после флага **-O/--opt**, так как в этом случае предполагается оптимизация.

По умолчанию компиляция выполняется до текущего этапа проекта, а вывод осуществляется в стандартный поток вывода, если не указан другой вывод с помощью **-o/--output**. Все сообщения об ошибках выводятся в стандартный поток ошибок.

По умолчанию оптимизация не выполняется. Список названий оптимизаций будет представлен на этапах оптимизации.

Для каждого предоставленного языкового каркаса мы предоставили код, которого достаточно для реализации этого интерфейса. Он также возвращает список непонятных аргументов, которые могут быть использованы для добавления функций. TAs не будет использовать какие-либо дополнительные функции, которые вы добавляете для выставления оценок. Однако вы можете указать нам, какие из них, если таковые имеются, следует использовать для компилятора derby. Возможно, вы захотите указать флаг, который будет включать только те оптимизации, которые вам нравятся.

7 Проектная документация

Начиная с третьего этапа, вы должны добавлять проектный документ в папку **doc** вашего репозитория **GitHub** в формате **PDF**. Документация должна быть понятной, краткой и легко читаемой.

Ваша документация должна включать следующие разделы, которые можно условно назвать «Разработка», «Дополнения», «Трудности» и «Вклад». Не обязательно отвечать на каждый вопрос или пункт в каждом разделе, достаточно предоставить информацию, которая позволит эффективно описать каждую часть.

Если в процессе реализации вы обнаружите недостатки в своем проекте, обсудите их и подумайте, что можно было бы сделать по-другому. Также расскажите о любых изменениях, которые вы внесли в предыдущие части, и объясните, почему они были необходимы.

Этот раздел должен помочь преподавателю научиться читать код и давать обратную связь по написанному коду.

8 Дополнительно

Список любых уточнений, допущений или дополнений к поставленной задаче. Сюда входят любые интересные методы отладки/логирования, дополнительные скрипты сборки или одобренные библиотеки, использованные при разработке компилятора. Спецификации проекта довольно обширны и оставляют на ваше усмотрение многие решения. Это неотъемлемая часть настоящей программной инженерии. Если вы считаете, что необходимы серьезные уточнения, обратитесь к преподавателю.

9 Трудности

Список известных проблем, связанных с вашим проектом, и все, что вам известно об их причинах.

- Если ваш проект не проходит один из тестовых сценариев, но вы не можете устранить проблему, опишите, в чем, по вашему мнению, заключается проблема.
- Если в ходе собственного тестирования вы обнаружили в проекте проблемы, которые не можете устранить, но которые не выявляются в рамках тестовых сценариев, опишите проблему как можно подробнее и укажите как можно больше информации о ее причине.
- Если на предыдущем этапе вы не проходили какие-то тестовые сценарии, но смогли исправить ошибки к текущему этапу, кратко опишите, в чем заключалась проблема и как вы ее устранили.
- Опишите любой раздел вашего проекта, который вы хотели бы выделить, чтобы получить больше отзывов или задать вопросы.

10 Вклад

Краткое описание того, как ваша группа распределила работу. Это не повлияет на вашу оценку, но поможет преподавателю выявить возможные проблемы.